

REPUBLIQUE DEMOCRATIQUE DU CONGO
UNIVERSITE DE KINSHASA



FACULTE DES SCIENCES ET TECHNOLOGIES
Mention Mathématiques, Statistique et Informatique
B.P. 190 KINSHASA XI

Master 2

LOGIQUE FLOUE EN INTELLIGENCE ARTIFICIELLE

**Système de Recommandation Flou base sur les Préférences
Imprecises des Utilisateurs**

Application au catalogue MovieLens via une inference de Mamdani

Sujet 12

DUASENGE MAYANO Brummel

Juin 2026

Résumé

Ce rapport présente la conception, l'implémentation et l'évaluation d'un système de recommandation de films fondé sur la logique floue. Le problème traité est celui des préférences utilisateur imprécises, souvent exprimées sous forme linguistique ou d'intervalle plutôt que par une note numérique unique. Le système propose de combiner un pré-filtrage par genre, une inference de Mamdani et une défuzzification par centroïde afin de produire un score interprétable pour chaque film candidat. L'application est réalisée sur le jeu de données MovieLens et exposée à travers une interface en ligne de commande et une interface graphique Tkinter.

Table des matières

1	Introduction	4
2	Cadre theorique	4
2.1	Logique floue et ensembles flous	5
2.2	Variables linguistiques	5
2.3	Preferences imprecises	5
2.4	Inference de Mamdani	6
2.5	Systemes de recommandation et MovieLens	6
3	Architecture du systeme	6
4	Modelisation floue	7
4.1	Variables linguistiques V1	7
4.2	Fonctions d'appartenance	8
4.3	Base de regles minimale	10
4.4	Agregation et defuzzification	10
5	Modelisation des preferences imprecises	11
5.1	Trois modes de saisie	11
5.2	Projection vers des degres d'appartenance	11
5.3	Agregation multi-genres	11
6	Pipeline de recommandation	12
7	Implementation	13
7.1	Stack technologique	13
7.2	Organisation du code	13
7.3	Configuration externalisee	14
7.4	Execution	14
8	Interfaces utilisateur	14
8.1	Interface CLI	14
8.2	Interface GUI	15
9	Resultats et evaluation	18
9.1	Protocole experimental	18
9.2	Exemple de recommandation detaillee	18
9.3	Illustration de l'inference Mamdani	19
9.4	Metriques d'evaluation	19
9.5	Tests automatises	20
10	Discussion, conclusion et perspectives	20
10.1	Discussion	20
10.2	Adequation au sujet	20
10.3	Perspectives	21

10.4 Conclusion	21
A Code source	23

1 Introduction

Les catalogues numeriques de films, de musiques ou de produits ont atteint une taille qui rend la recherche manuelle peu praticable. Dans ce contexte, les systemes de recommandation jouent un role central : ils filtrent un espace de choix tres large afin de proposer quelques items compatibles avec le profil d'un utilisateur. Toutefois, les preferences humaines sont rarement exprimees sous une forme parfaitement numerique. Un utilisateur peut dire qu'il aime beaucoup la science-fiction, qu'il apprecie moyennement les films d'action, ou qu'il hesite entre plusieurs niveaux d'interet. Ces expressions sont naturelles, mais elles introduisent une imprecision que les modeles classiques traitent souvent par une conversion brutale en notes crisp.

La logique floue fournit un cadre adapte a ce probleme, car elle permet de represente des degres d'appartenance entre 0 et 1 plutot qu'une appartenance binaire [6, 8]. Dans un systeme de recommandation, cette propriete permet d'associer un film a des termes tels que *bonne note*, *forte popularite* ou *preference moyenne*, puis de combiner ces informations dans une base de regles interpretable. L'enjeu n'est donc pas seulement de produire un classement, mais de conserver une trace comprehensible du raisonnement.

La problematique etudiee dans ce rapport est la suivante : comment represente et traiter computationnellement l'imprecision intrinseque des preferences utilisateur afin de produire des recommandations interpretable et reproductibles ?

Les objectifs du travail sont les suivants :

1. modeliser les preferences imprecises a l'aide de la logique floue ;
2. concevoir un moteur d'inference de Mamdani dont les regles restent lisibles ;
3. appliquer le systeme au jeu de donnees MovieLens ;
4. fournir une interface en ligne de commande et une interface graphique permettant l' experimentation.

Le rapport est organise en dix sections. La section 2 presente le cadre theorique : ensembles flous, variables linguistiques, preferences imprecises, inference de Mamdani et donnees MovieLens. La section 3 decrit l'architecture logicielle. La section 4 detaille la modelisation floue retenue. La section 5 precise la representation des preferences imprecises. La section 6 presente le pipeline de recommandation. La section 7 expose les choix d'implementation. La section 8 decrit les interfaces utilisateur. La section 9 presente les resultats experimentaux. Enfin, la section 10 discute les forces, limites et perspectives du systeme.

La section suivante pose donc les bases theoriques necessaires pour comprendre le passage d'une preference vague a une decision de recommandation interpretable.

2 Cadre theorique

Cette section introduit les notions mobilisees dans le systeme developpe. Elle part de la definition d'un ensemble flou, puis montre comment les variables linguistiques et l'inference de Mamdani permettent de manipuler des preferences imprecises.

2.1 Logique floue et ensembles flous

Dans la theorie classique des ensembles, un element appartient ou n'appartient pas a un ensemble. La logique floue propose une generalisation : un ensemble flou $\tilde{A}$ defini sur un univers X est caracterise par une fonction d'appartenance

$$\mu_A : X \rightarrow [0, 1],$$

ou $\mu_A(x)$ mesure le degre auquel x appartient a $\tilde{A}$ [4, 8]. Ainsi, une note moyenne de film peut appartenir partiellement au terme *bonne* et partiellement au terme *excellente*.

Deux formes simples sont utilisees dans ce projet. Une fonction triangulaire de parametres (a, b, c) est donnee par :

$$\mu(x) = \begin{cases} 0, & x \leq a, \\ \frac{x-a}{b-a}, & a < x \leq b, \\ \frac{c-x}{c-b}, & b < x < c, \\ 0, & x \geq c. \end{cases}$$

Une fonction trapezoidale de parametres (a, b, c, d) est donnee par :

$$\mu(x) = \begin{cases} 0, & x \leq a, \\ \frac{x-a}{b-a}, & a < x \leq b, \\ 1, & b < x \leq c, \\ \frac{d-x}{d-c}, & c < x < d, \\ 0, & x \geq d. \end{cases}$$

Ces fonctions sont simples, mais suffisantes pour obtenir des zones de transition progressives entre termes voisins.

2.2 Variables linguistiques

Une variable linguistique associe un univers numerique a un ensemble de termes interpretables [9]. Par exemple, la variable **genre_preference** est definie sur $[0, 1]$ avec les termes *faible*, *moyenne* et *forte*. La granularite choisie influence directement la lisibilite des regles : peu de termes rendent le systeme compact, tandis qu'un nombre plus eleve de termes permet une decision plus fine.

Dans ce travail, les variables linguistiques ne sont pas seulement descriptives. Elles sont les entrees et la sortie du moteur Mamdani : preference de genre, note moyenne, popularite et score de recommandation. Ce lien entre langage et calcul ouvre naturellement la voie a la representation des preferences imprecises.

2.3 Preferences imprecises

Une preference imprecise designe une information utilisateur dont la valeur exacte n'est pas connue ou n'est pas exprimee sous forme ponctuelle. Le systeme supporte trois modes. Le mode linguistique permet de saisir un terme comme *forte*. Le mode intervalle permet de saisir une plage telle que 0.6..0.9. Le mode crisp accepte une valeur numerique unique dans $[0, 1]$ lorsque l'utilisateur veut etre precis.

Pour relier ces modes au calcul flou, le systeme utilise une projection vers des degres d'appartenance. Un terme linguistique est transforme en zone d'incertitude par un alpha-cut a 0.5 ; un intervalle est projete en prenant, pour chaque terme, le degre maximal atteint dans la plage. Cette strategie evite de reduire immediatement une expression vague a un seul nombre.

2.4 Inference de Mamdani

L'inference de Mamdani est un modele de raisonnement flou fonde sur des regles de la forme *si... alors...* [5]. Dans ce projet, le processus suit quatre etapes : fuzzification des entrees, activation des regles, agregation des sorties et defuzzification.

Pour une regle comportant plusieurs antecedents, le degre d'activation est calcule par un operateur minimum. Les sorties activees sont ensuite agregees par maximum. Enfin, le score crisp est obtenu par la methode du centroide :

$$z^* = \frac{\int z \cdot \mu(z) dz}{\int \mu(z) dz}.$$

Ce choix conserve une interpretation graphique claire : le score final correspond au centre de gravite de la surface floue agregee.

2.5 Systemes de recommandation et MovieLens

MovieLens est un ensemble de donnees de reference pour l'etude des systemes de recommandation [2]. La version utilisee dans ce projet contient quatre fichiers principaux : `movies.csv`, `ratings.csv`, `tags.csv` et `links.csv`. Les films apportent les titres et genres, les notes permettent de calculer des moyennes et des popularites, les tags donnent des annotations libres, et les liens fournissent des identifiants externes.

Le projet exploite principalement les genres, les notes moyennes et le nombre de notes. Ces informations sont suffisantes pour construire une premiere version interpretable du systeme. La section suivante montre comment elles sont organisees dans une architecture logicielle modulaire.

3 Architecture du systeme

L'architecture du systeme separe clairement les responsabilites : chargement des donnees, calcul flou, recommandation, evaluation, visualisation et interfaces. Cette separation facilite les tests et rend le comportement du moteur plus reproductible.

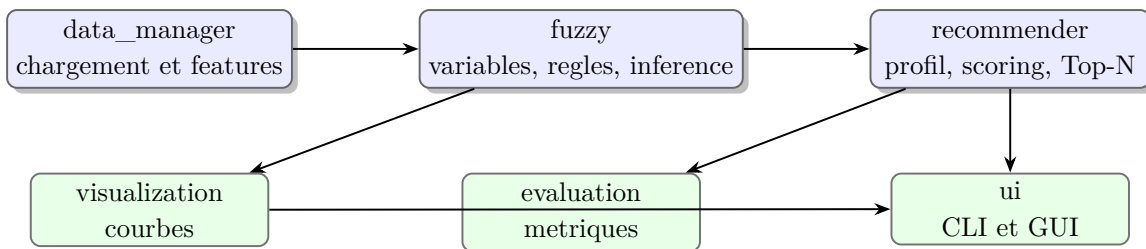


FIGURE 1 – Architecture modulaire du systeme FuzzyRec.

Le module `data_manager` lit les fichiers MovieLens, valide les tables et construit les caracteristiques de films. Le module `fuzzy` contient les variables linguistiques, les fonctions d'appartenance, la base de regles, l'inference de Mamdani et la defuzzification. Le module `recommender` assemble ces services pour produire les recommandations. Les modules `evaluation`, `visualization` et `ui` fournissent respectivement les metriques, les figures et les interfaces.

TABLE 1 – Responsabilites principales des modules.

Module	Responsabilite	Fichiers cles
<code>data_manager</code>	Chargement et preprocessing MovieLens	<code>loader.py</code> , <code>preprocessor.py</code>
<code>fuzzy</code>	Representation floue et Mamdani	<code>inference_engine.py</code> , <code>rule_base.py</code>
<code>recommender</code>	Profil, prefiltrage, score Top-N	<code>fuzzy_recommender.py</code> , <code>pipeline_factory.py</code>
<code>evaluation</code>	Precision, rappel, couverture, diversite	<code>metrics.py</code> , <code>evaluator.py</code>
<code>visualization</code>	Courbes d'appartenance	<code>membership_plots.py</code>
<code>ui</code>	CLI et interface Tkinter	<code>commands.py</code> , <code>main_window.py</code>

Cette organisation prepare la modelisation floue proprement dite : les variables et les regles peuvent etre decrites independamment des interfaces qui les exploitent.

4 Modelisation floue

La modelisation floue V1 repose sur quatre variables linguistiques et une base minimale de huit regles. Les definitions sont externalisees dans `config/fuzzy_config.yaml`, ce qui rend le systeme inspectable et ajustable sans modifier le code Python.

4.1 Variables linguistiques V1

TABLE 2 – Variables linguistiques de la V1.

Variable	Univers	Termes
<code>genre_preference</code>	[0, 1]	faible, moyenne, forte
<code>average_rating</code>	[0.5, 5]	mauvaise, correcte, bonne, excellente
<code>popularity</code>	[0, 350]	confidentiel, modere, populaire, tres populaire
<code>recommendation_score</code>	[0, 1]	tres faible, faible, moyen, fort, tres fort

Les trois premieres variables sont les entrees du systeme Mamdani. La quatrieme est la variable de sortie defuzzifiee pour obtenir le score final.

4.2 Fonctions d'appartenance

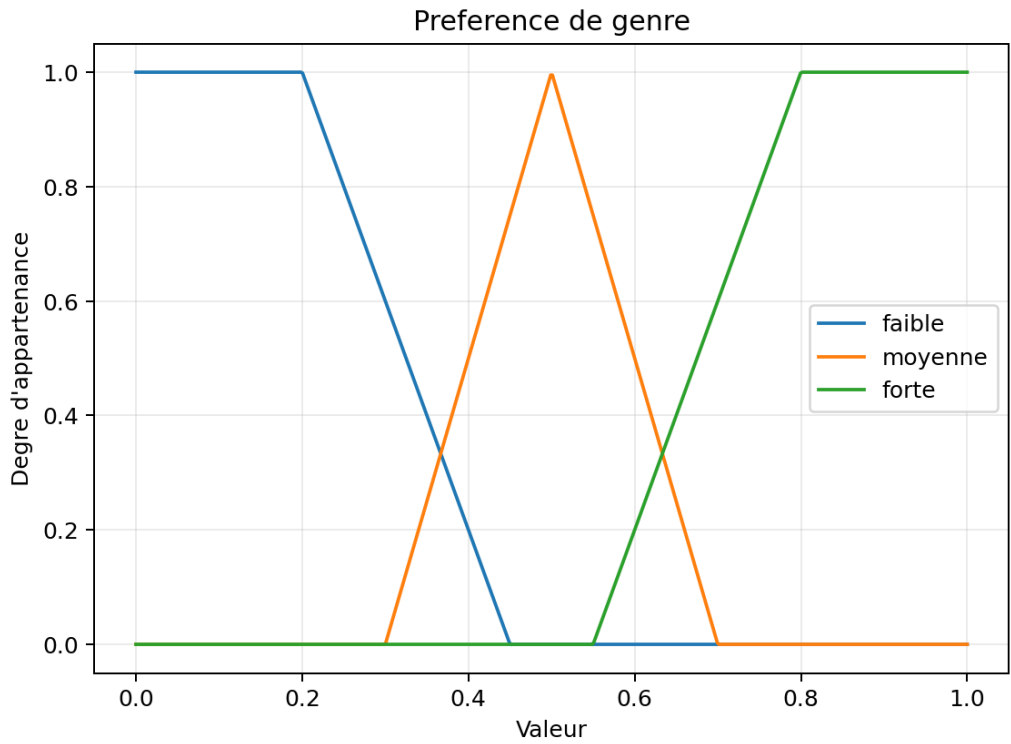


FIGURE 2 – Fonctions d'appartenance de la preference de genre.

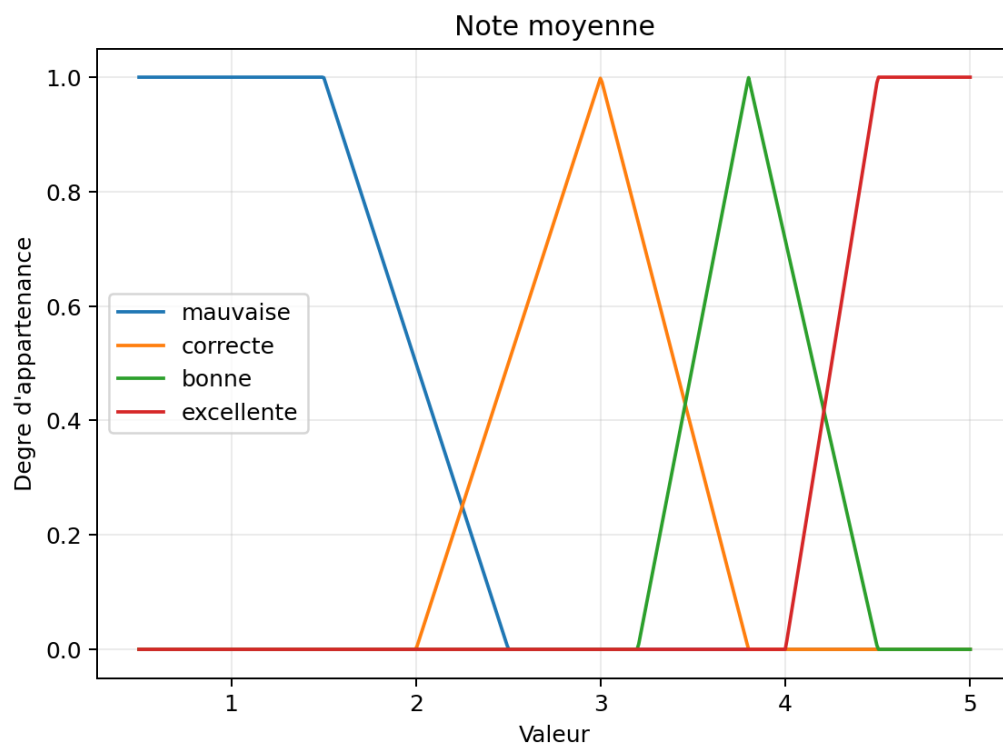


FIGURE 3 – Fonctions d'appartenance de la note moyenne.

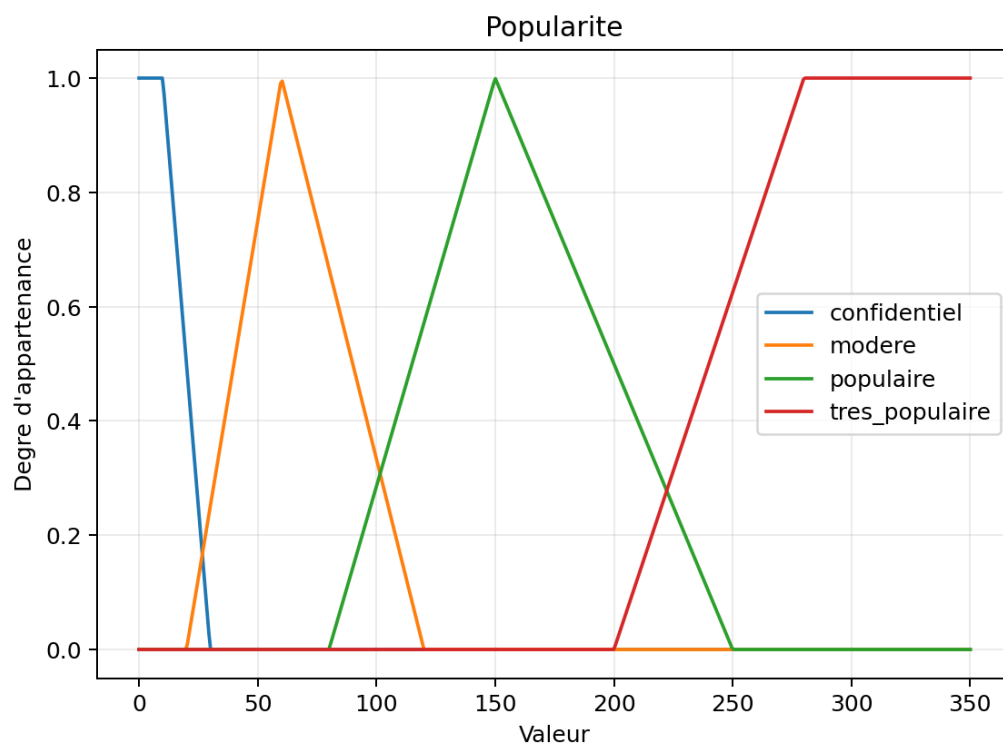


FIGURE 4 – Fonctions d'appartenance de la popularite.

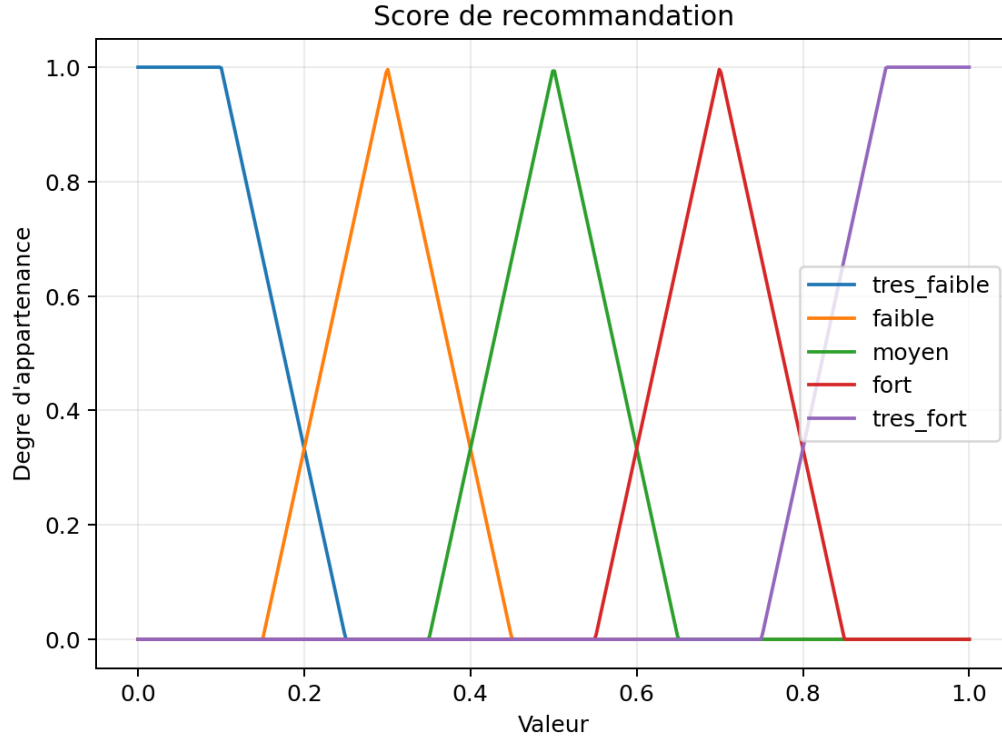


FIGURE 5 – Fonctions d'appartenance du score de recommandation.

4.3 Base de regles minimale

TABLE 3 – Base de regles Mamdani minimale.

Regle	Antecedents	Consequence
R1	preference forte, note excellente, popularite tres populaire	tres fort
R2	preference forte, note excellente, popularite confidentiel	fort
R3	preference forte, note bonne, popularite populaire	fort
R4	preference moyenne, note excellente, popularite tres populaire	fort
R5	preference moyenne, note bonne, popularite modere	moyen
R6	preference faible, note excellente, popularite tres populaire	moyen
R7	preference forte, note mauvaise, popularite populaire	faible
R8	preference faible, note mauvaise, popularite confidentiel	tres faible

Cette base de regles est volontairement compacte. Elle suffit a demontrer le raisonnement flou tout en gardant chaque decision explicable.

4.4 Agregacion et defuzzification

Le moteur Mamdani utilise le minimum pour combiner les antecedents d'une regle, puis le maximum pour agreger les sorties activees. La defuzzification est realisee par centroide, conformement au parametrage `defuzzification_method: centroid`. La figure d'inference de la section 9 illustre cette surface agregee et la position du centroide.

La section suivante precise comment les preferences utilisateur, souvent imprecises, sont projetees vers ces variables linguistiques.

5 Modelisation des preferences imprecises

Le coeur du sujet porte sur la maniere de represente une preference qui n'est pas parfaitement numerique. Le systeme distingue explicitement la saisie utilisateur de son exploitation par le moteur flou.

5.1 Trois modes de saisie

Le profil utilisateur accepte trois formes. La premiere est linguistique, par exemple `Sci-Fi=forte`, et correspond a `LinguisticGenrePreference`. La deuxieme est un intervalle, par exemple `Action=0.5..0.8`, represente par `IntervalGenrePreference`. La troisieme est crisp, c'est-a-dire une valeur ponctuelle dans $[0, 1]$.

Dans la version actuelle, les valeurs crisp sont acceptees seulement lorsque ce mode est choisi explicitement. Cette contrainte evite de diluer l'objectif du projet, qui est de mettre en avant les preferences imprecises plutot qu'une simple notation numerique.

5.2 Projection vers des degres d'appartenance

La fonction `Fuzzifier.fuzzify_imprecise_value` realise la transformation vers les degres flous. Pour un terme linguistique, le systeme calcule l'alpha-cut a 0.5 du terme correspondant, puis projette cet intervalle sur toutes les fonctions d'appartenance de la variable. Pour un intervalle explicite, la projection prend le maximum atteint par chaque terme sur la plage. Pour une valeur crisp, la fuzzification classique est appliquee.

Par exemple, une preference `forte` n'est pas remplacee par une constante unique. Elle active la zone du terme *forte* et peut conserver un chevauchement avec le terme voisin. De meme, l'intervalle `0.5..0.8` peut activer simultanement *moyenne* et *forte*. Cette approche rejoint l'idee generale de la logique floue : l'incertitude est propagee plutot qu'effacee.

5.3 Agregation multi-genres

Un film `MovieLens` peut appartenir a plusieurs genres. La methode `genre_preference_for_movie` compare donc les genres du film aux preferences declarees dans le profil. Si aucun genre ne correspond, le systeme retourne un etat *inconnu*, distinct d'une preference faible. Si plusieurs genres correspondent, la strategie par defaut calcule la moyenne des forces de preference. Une strategie par maximum reste disponible, mais elle est moins nuancee, car elle peut masquer une preference faible associee a un autre genre. Une extension naturelle consisterait a utiliser des operateurs OWA pour controler plus finement l'optimisme ou la prudence de cette aggregation [7].

Ainsi, avec un profil `{Sci-Fi : forte, Action : 0.5..0.8}`, un film *Action/Sci-Fi* combine une preference linguistique forte et un intervalle moyen-fort. Le systeme conserve cette nuance jusqu'a la fuzzification, puis active les regles Mamdani selon les degres obtenus.

Une fois les preferences representees, il reste a organiser leur passage dans la chaine complete de recommandation. C'est l'objet de la section suivante.

6 Pipeline de recommandation

Le pipeline transforme un profil utilisateur et un catalogue de films en liste Top-N expliquée. Il est implémenté principalement dans `pipeline_factory.py`, `fuzzy_recommender.py` et `explanation_engine.py`.

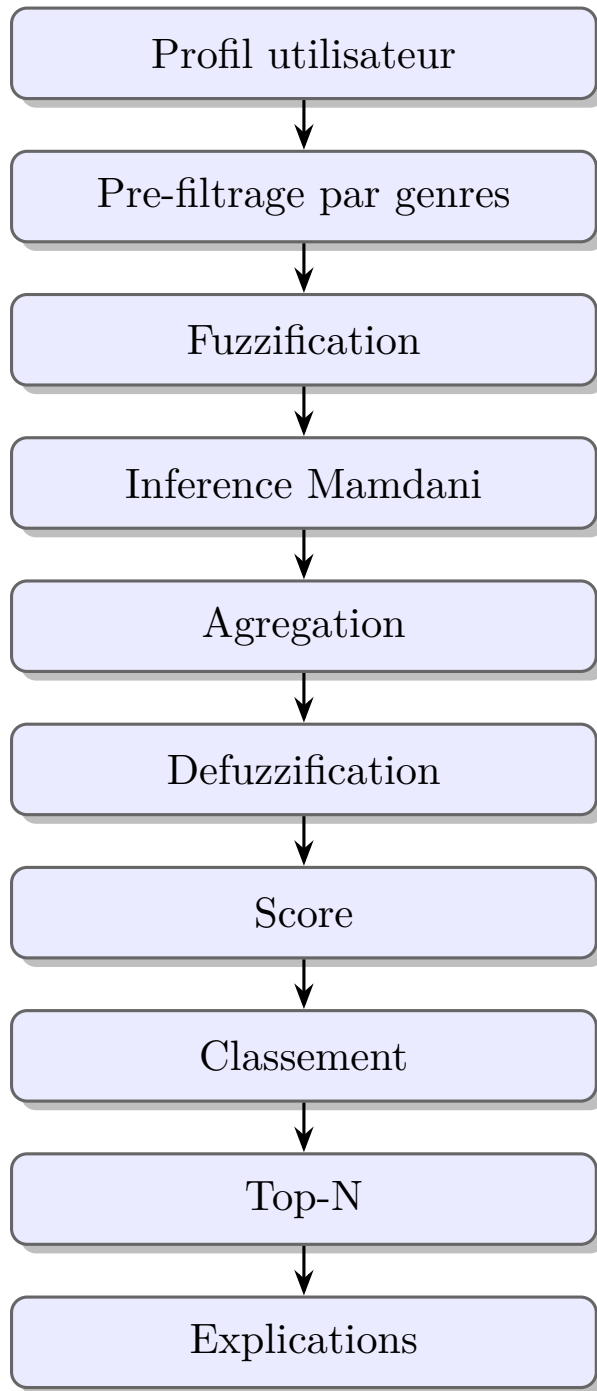


FIGURE 6 – Pipeline vertical de recommandation.

La première étape construit le profil. Si des préférences explicites sont fournies, elles priment

sur l'historique MovieLens. Sinon, le profil est derive des notes de l'utilisateur par genre. Cette construction est centralisee dans `build_profile`.

La deuxieme etape effectue un pre-filtrage par genre. Le seuil `V1` est fixe a 0.2 dans la configuration. Les films sans popularite connue sont exclus du Top-N afin d'eviter qu'une valeur manquante soit interpretee comme une faible popularite reelle.

La troisieme etape calcule les entrees crisp ou imprecises du moteur flou : preference de genre, note moyenne et popularite. Ces valeurs sont fuzzifiees selon les variables linguistiques de la `V1`.

La quatrieme etape active la base de regles Mamdani. Chaque regle produit un degre d'activation et une consequence sur la variable de sortie. Les sorties sont agregees par maximum, puis defuzzifiees par centroide pour produire un score dans $[0, 1]$.

Enfin, les films sont tries par score decroissant, puis par popularite, note moyenne et titre. L'objet de recommandation conserve aussi les entrees floues et les regles activees, ce qui permet a `ExplanationEngine` de generer une trace lisible.

Ce pipeline etant defini, la section suivante presente les choix techniques qui ont permis de l'implementer et de le tester.

7 Implementation

L'implementation vise deux objectifs : rester proche du modele theorique et offrir une base testable. Les choix techniques sont donc simples, largement disponibles et adaptes a une execution locale.

7.1 Stack technologique

Le projet est implemente en Python. Le fichier `pyproject.toml` declare une compatibilite Python ≥ 3.10 . Les principales dependances sont `pandas` pour la manipulation des tables MovieLens, `NumPy` pour les calculs numeriques, `PyYAML` pour la configuration, `Tkinter` pour la GUI, `Matplotlib` pour les figures et `pytest` pour les tests. Le paquet `click` fournit l'interface CLI.

7.2 Organisation du code

L'arborescence du dossier `src/` suit les modules presentes dans l'architecture :

```
src/
data_manager/    chargement, validation, preprocessing
fuzzy/           ensembles flous, variables, Mamdani, defuzzification
recommender/     profils, prefiltrage, scoring, explications
evaluation/      metriques et protocole train/test
ui/              commandes CLI et interface Tkinter
visualization/   generation des courbes d'appartenance
```

Cette organisation permet de tester separement les composants. Par exemple, la defuzzification peut etre verifiee sans lancer l'interface graphique, et l'evaluation peut reutiliser un moteur deja construit.

7.3 Configuration externalisee

Le fichier `config/fuzzy_config.yaml` contient les variables linguistiques, les fonctions d'appartenance, les regles, la methode de defuzzification, le seuil de pre-filtrage et les chemins de donnees. Cette externalisation ameliore la reproductibilite : les valeurs du rapport peuvent etre relancees avec le meme fichier de configuration. Elle facilite aussi les ajustements : ajouter des regles ou modifier une fonction d'appartenance ne demande pas de recompilation ni de modification des interfaces.

7.4 Execution

Les commandes principales sont les suivantes :

```
python main.py recommend --user-id 1 --top-n 10 \
    --set-genre "Sci-Fi=forte,Action=moyenne" --explain
```

```
python main.py gui --data-dir data/movie
```

```
python main.py evaluate --user-id 1 --top-n 10
```

La premiere commande produit un Top-N explique, la deuxieme lance l'interface graphique, et la troisieme calcule les metriques d'evaluation. Ces commandes constituent le point d'entree des experiences de la section 9.

La section suivante presente les interfaces qui rendent ces operations accessibles a l'utilisateur.

8 Interfaces utilisateur

Deux interfaces sont proposees. La CLI sert a reproduire rapidement les experiences et a automatiser les tests. La GUI sert a manipuler les preferences de maniere visuelle et a observer les courbes d'appartenance.

8.1 Interface CLI

L'interface en ligne de commande est exposee par `main.py`. Elle regroupe notamment les sous-commandes `recommend`, `evaluate`, `dataset-stats` et `gui`. La commande `recommend` accepte un identifiant utilisateur, un nombre de recommandations et des preferences explicites. Elle peut aussi afficher l'explication des regles activees.

```

Top-10 recommandations pour user_id=1
1. 260 | 0.8917 | Star Wars: Episode IV - A New Hope (1977)
2. 2571 | 0.8891 | Matrix, The (1999)
3. 1196 | 0.7418 | Star Wars: Episode V - The Empire Strikes Back (1980)
4. 1580 | 0.7000 | Men in Black (a.k.a. MIB) (1997)
5. 5349 | 0.7000 | Spider-Man (2002)
6. 1206 | 0.7000 | Clockwork Orange, A (1971)
7. 2716 | 0.7000 | Ghostbusters (a.k.a. Ghost Busters) (1984)
8. 5445 | 0.7000 | Minority Report (2002)
9. 1356 | 0.7000 | Star Trek: First Contact (1996)
10. 589 | 0.7000 | Terminator 2: Judgment Day (1991)

Explication #1
Film : Star Wars: Episode IV - A New Hope (1977)
Score de recommandation : 0.8917 (fortement recommande, sortie dominante: tres fort)
Pourquoi ce film est recommande :
- preference de genre = forte (mu=0.8000, valeur=0.75)
- note moyenne = excellente (mu=0.4622, valeur=4.231075697211155)
- popularite = tres populaire (mu=0.6375, valeur=251.0)
Regles floues actives :
- R1 [mu=0.4622] : preference de genre=forte (mu=0.8000) ET note moyenne=excellente (mu=0.4622) ET popularite=tres populaire (mu=0.6375) -> tres fort
Defuzzification centroide : 0.8917

Explication #2
Film : Matrix, The (1999)
Score de recommandation : 0.8891 (fortement recommande, sortie dominante: tres fort)
Pourquoi ce film est recommande :
- preference de genre = forte (mu=0.8000, valeur=0.75)
- note moyenne = bonne (mu=0.4394, valeur=4.192446043165468)
- popularite = tres populaire (mu=0.9750, valeur=278.0)
Regles floues actives :
- R1 [mu=0.3849] : preference de genre=forte (mu=0.8000) ET note moyenne=excellente (mu=0.3849) ET popularite=tres populaire (mu=0.9750) -> tres fort
Defuzzification centroide : 0.8891

```

FIGURE 7 – Capture de la commande CLI `recommend` pour `user_id=1`. La sortie affiche le Top-N, le score defuzzifie et une trace textuelle des regles Mamdani activees pour les premieres recommandations.

La CLI est particulierement utile pour verifier la reproductibilite des resultats, car les memes parametres peuvent etre relances sans interaction graphique.

8.2 Interface GUI

L'interface graphique Tkinter est organisee en trois zones. La premiere zone permet de choisir l'utilisateur, le nombre de recommandations et le dossier MovieLens. Le bouton *Changer dossier* permet de charger un autre dataset compatible. La deuxieme zone contient un notebook avec les onglets *Preferences* et *Courbes*. La troisieme zone affiche les films recommandes et la trace d'explication.

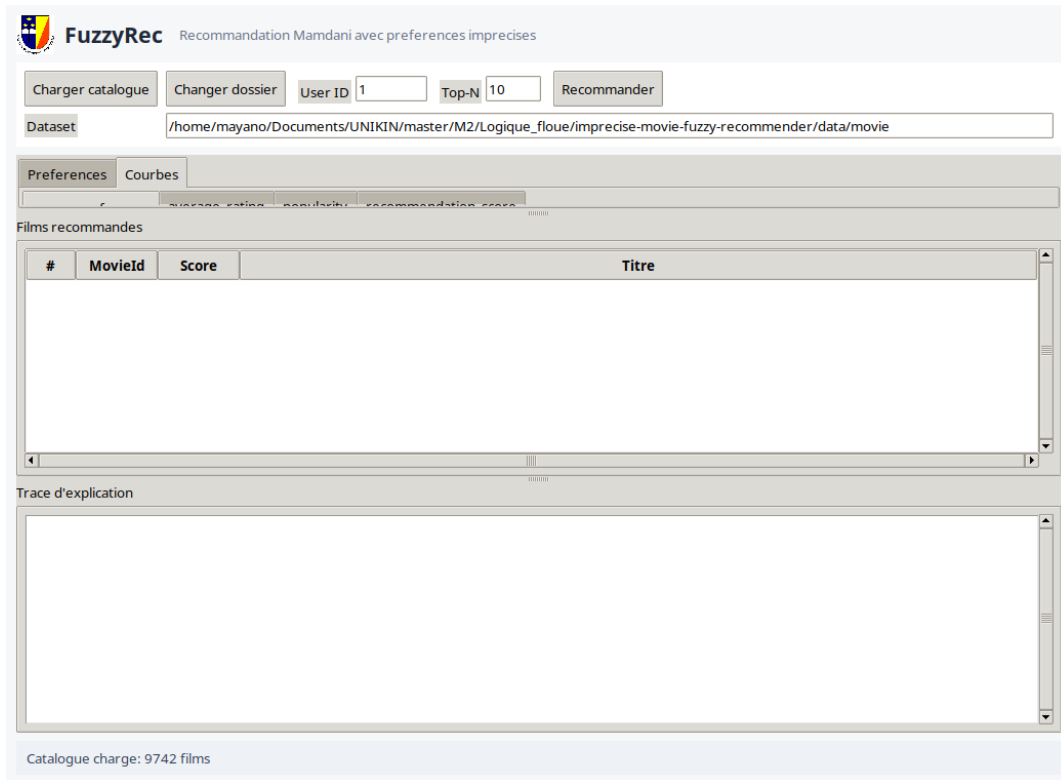


FIGURE 9 – Onglet des courbes : les fonctions d'appartenance des variables linguistiques sont integrees dans la GUI afin de rendre visible la transformation des preferences.

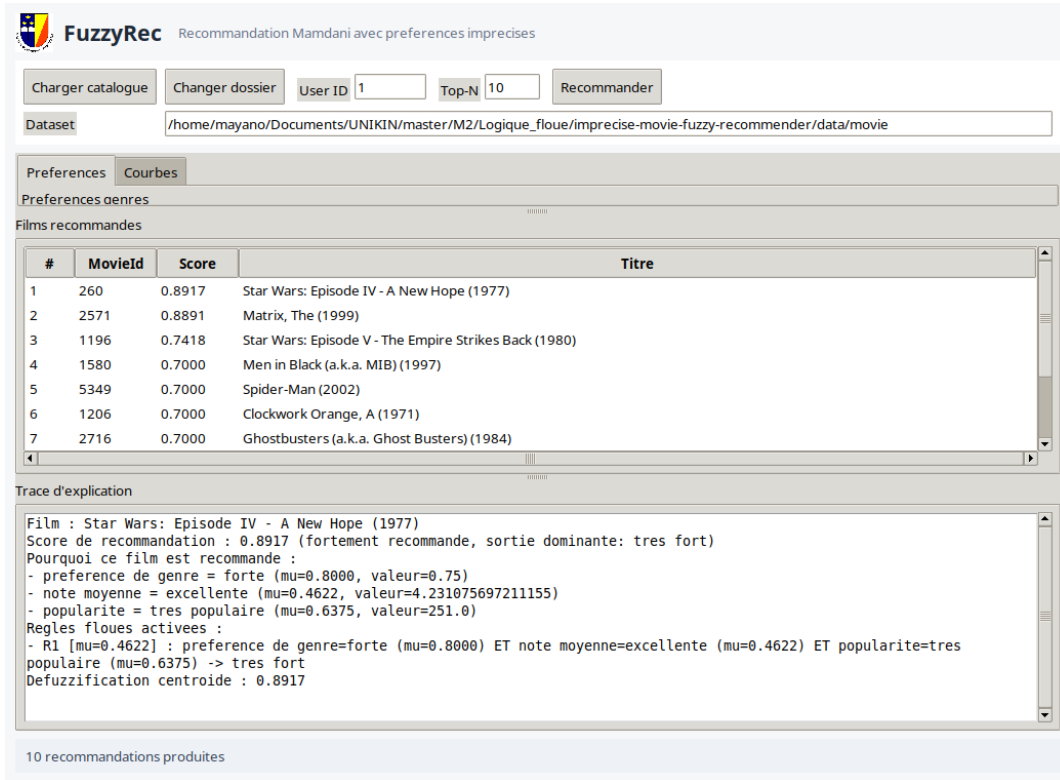


FIGURE 10 – Zone des resultats : le Top-N est affiche dans un tableau et la trace d’explication detaille les degres flous et les regles actives pour la recommandation selectionnee.

Deux problemes UX initiaux ont ete corriges. D’abord, la saisie par texte a ete remplacee par des curseurs par thematique, ce qui rend les preferences plus directes. Ensuite, la coherence entre les modes numerique, linguistique et intervalle est assuree par une combobox par genre. Ainsi, la GUI ne force pas l’utilisateur a connaitre la syntaxe CLI pour experimenter.

Ces interfaces permettent maintenant de produire les resultats experimentaux presentes dans la section suivante.

9 Resultats et evaluation

Cette section presente des resultats issus d’executions reelles du projet. Les commandes ont ete lancees sur le dossier `data/movie`, qui correspond au jeu MovieLens small utilise dans le projet.

9.1 Protocole experimental

Le protocole utilise le dataset MovieLens `ml-latest-small` [2]. Le profil d’exemple est `user_id=1`, avec les preferences explicites `Sci-Fi=forte`, `Action=moyenne`. Le nombre de recommandations est fixe a 5 pour l’exemple detaille et a 10 pour l’evaluation. Le seuil de pre-filtrage est celui de la configuration V1, soit 0.2.

9.2 Exemple de recommandation detaillee

La commande executee est :

```
python main.py recommend --user-id 1 --top-n 5 \
--set-genre "Sci-Fi=forte,Action=moyenne" --explain --data-dir data/movie
```

Le Top-5 obtenu est presente dans le tableau suivant.

TABLE 4 – Top-5 obtenu pour `user_id=1`.

Rang	Titre	Score
1	Star Wars : Episode IV - A New Hope (1977)	0.8917
2	Matrix, The (1999)	0.8891
3	Star Wars : Episode V - The Empire Strikes Back (1980)	0.7418
4	Men in Black (a.k.a. MIB) (1997)	0.7000
5	Spider-Man (2002)	0.7000

Pour la premiere recommandation, la trace indique une preference de genre forte avec $\mu = 0.8000$, une note moyenne excellente avec $\mu = 0.4622$ et une popularite tres populaire avec $\mu = 0.6375$. La regle R1 est activee avec un degre 0.4622 et produit la sortie *tres fort*. La defuzzification par centroide donne le score 0.8917.

9.3 Illustration de l'inference Mamdani

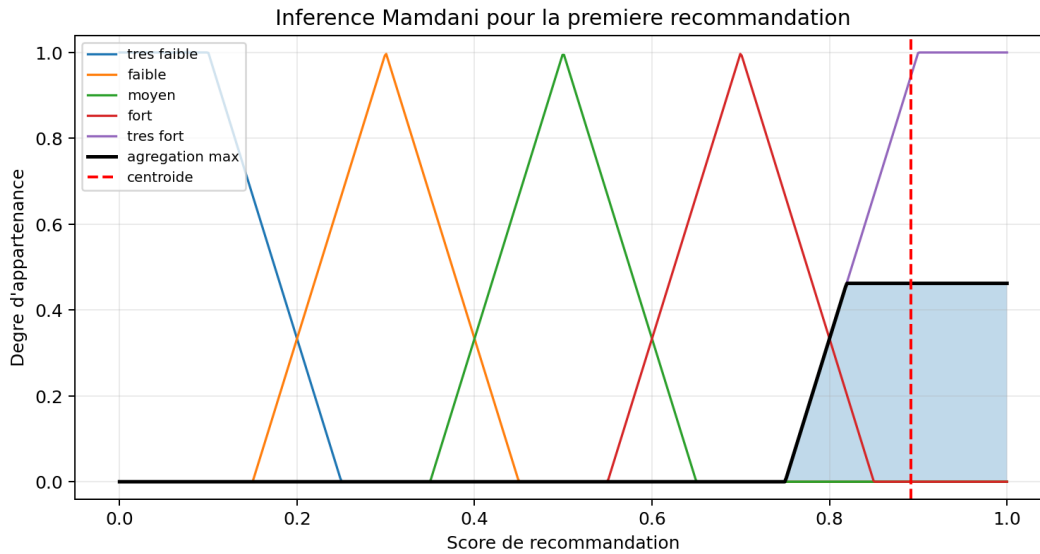


FIGURE 11 – Illustration de l'inference Mamdani pour la premiere recommandation : les sorties tronquees sont agregees par maximum, puis le centroide fournit le score final.

La figure montre que la recommandation ne depend pas d'une seule condition. Elle resulte de la combinaison des degres d'appartenance et de l'agregation des consequences floues.

9.4 Metriques d'evaluation

Le protocole `evaluate` realise un decoupage temporel des notes de l'utilisateur. Les notes anciennes construisent le profil, tandis que les notes recentes servent de reference. Les resultats suivants

proviennent de trois executions reelles avec `top-n=10`.

TABLE 5 – Metriques d’evaluation pour trois utilisateurs MovieLens.

Utilisateur	Precision@10	Recall@10	Coverage	Diversite
1	0.2000	0.0526	0.0010	0.7746
2	0.0000	0.0000	0.0010	0.7746
3	0.0000	0.0000	0.0010	0.6589

Ces valeurs doivent etre interpretees comme une verification experimentale de la chaine de calcul plutot que comme une optimisation finale. La base de regles V1 est volontairement minimale, ce qui limite la couverture decisionnelle.

9.5 Tests automatises

La commande `./venv/bin/pytest -q` a ete executee avant la redaction finale. Le resultat obtenu est :

`96 passed in 7.79s`

Les tests couvrent les fonctions d’appartenance, la fuzzification, la base de regles, l’inference, la defuzzification, le repository MovieLens, le pipeline de recommandation, les explications, les metriques, la CLI et les composants GUI.

Les resultats confirment que la V1 est executable et testee. La section suivante discute ce que cette version demontre, mais aussi ce qu’elle ne pretend pas encore resoudre.

10 Discussion, conclusion et perspectives

10.1 Discussion

La principale force du systeme est son interpretabilite. Chaque recommandation conserve les degres d’appartenance, les regles activees et le score defuzzifie. Cette trace permet d’expliquer pourquoi un film est classe en tete, ce qui est plus transparent qu’un simple score opaque.

Le deuxieme apport est la prise en charge de plusieurs modes d’expression des preferences : linguistique, intervalle et crisp. Le systeme separe la representation imprecise de la decision finale. La preference n’est donc pas immediatement reduite a une note unique ; elle est projete vers des degres flous et combinee par le moteur Mamdani.

Les limites sont egalement claires. La base V1 contient huit regles seulement. Elle suffit pour demontrer le principe, mais ne couvre pas toutes les situations possibles. Le pre-filtrage par genre reste simple et ne tient pas encore compte des tags ou de la recence. Enfin, la popularite est utilisee comme critere global, sans personnalisation profonde selon le comportement de chaque utilisateur.

10.2 Adequation au sujet

Le sujet 12 demande un systeme de recommandation flou base sur les preferences imprecises des utilisateurs [3]. Le systeme repond a ce sujet sur quatre points. Premièrement, il represente explicitement les preferences imprecises par termes linguistiques et intervalles. Deuxièmement, il

utilise un moteur Mamdani interpretable pour convertir ces preferences en score de recommandation. Troisiemement, il applique le modele a un catalogue reel, MovieLens. Quatriemement, il fournit une CLI et une GUI permettant de reproduire les resultats et de visualiser les memberships.

10.3 Perspectives

La premiere perspective consiste a etendre la base de regles. Le fichier YAML contient deja une entree **intermediate** prevue pour une V2 plus riche. La deuxieme perspective est l'integration des tags MovieLens, qui permettraient de modeliser des preferences thematiques plus fines que les genres. Une representation semantique floue, proche des travaux sur les ontologies floues [1], pourrait alors relier genres, mots-cles et concepts narratifs. La troisieme perspective concerne la recence : un film ancien et un film recent pourraient etre traites differemment selon le profil. Enfin, les parametres des fonctions d'appartenance pourraient etre appris ou ajustes automatiquement a partir des historiques utilisateurs.

10.4 Conclusion

Ce travail a montre qu'un systeme de recommandation peut traiter des preferences vagues sans les convertir immediatement en notes rigides. En combinant variables linguistiques, preferences imprecises, inference de Mamdani et explications, la solution proposee fournit une V1 coherent, interpretable et reproductible sur MovieLens. Les resultats experimentaux restent modestes, mais ils valident l'objectif principal : construire une chaine complete reliant l'expression naturelle de la preference a une recommandation floue expliquee.

Références

- [1] Fernando Bobillo and Umberto Straccia. Fuzzy ontology representation using owl 2. *International Journal of Approximate Reasoning*, 52(7) :1073–1094, 2011.
- [2] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets : History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4) :1–19, 2015.
- [3] Selain Kasereka. Logique floue en intelligence artificielle. Notes de cours, M2 Informatique, Universite de Kinshasa, 2026.
- [4] George J. Klir and Bo Yuan. *Fuzzy Sets and Fuzzy Logic : Theory and Applications*. Prentice Hall, 1995.
- [5] Ebrahim H. Mamdani and Sedrak Assilian. An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1) :1–13, 1975.
- [6] Timothy J. Ross. *Fuzzy Logic with Engineering Applications*. Wiley, 3 edition, 2010.
- [7] Ronald R. Yager. On ordered weighted averaging aggregation operators in multicriteria decisionmaking. *IEEE Transactions on Systems, Man, and Cybernetics*, 18(1) :183–190, 1988.
- [8] Lotfi A. Zadeh. Fuzzy sets. *Information and Control*, 8(3) :338–353, 1965.
- [9] Lotfi A. Zadeh. The concept of a linguistic variable and its application to approximate reasoning. *Information Sciences*, 8(3) :199–249, 1975.

A Code source

L'intégralité du code source, des tests et de la configuration est publiée sous licence libre dans le dépôt public suivant : <https://github.com/JudicaelMAKWIZA/imprecise-movie-fuzzy-recommender>. Le lecteur y trouvera les instructions d'installation, le jeu de données MovieLens utilisé et les scripts permettant de reproduire l'ensemble des résultats présentés dans ce rapport.